MACHINE LEARNING EXAM PAPER 2025

PLEASE ANSWER 3 OUT OF 4 QUESTIONS.

1. Linear Classification, Non-linear Feature Transformation & Perceptron Learning Algorithm

a) Hoeffding's inequality can be written as:

$$P\left(|\hat{R}_n(h) - R(h)| > \varepsilon\right) \leq 2e^{-2n\varepsilon^2}.$$

Answer the following:

i) Starting from Hoeffding's inequality, state the generalization bound that holds with probability at least $1-\delta$ for a fixed hypothesis h. Then, Derive an expression for the minimum required sample size n in terms of the tolerance ε and confidence parameter δ for a fixed hypothesis h. **[15 marks]**

Solution: For a fixed hypothesis h, the generalization bound can be expressed with probability at least $1-\delta$ as

$|\hat{R}_n(h) - R(h)| \leq \varepsilon$, where

$$\delta = 2e^{-2n\varepsilon^2}.$$

Solving for n yields

$$n = -\frac{\ln(\delta/2)}{2\varepsilon^2}.$$

ii) Using the generalization bound derived earlier, determine the minimum number of samples n required to ensure that

$$|\hat{R}_n(h) - R(h)| \leq 0.03$$

with probability at least 99% for a fixed hypothesis h. **[15 marks]**

Solution: Given $\varepsilon = 0.03$ and $1-\delta = 0.99 \Rightarrow \delta = 0.01$,

$$n = -\frac{\ln(0.01/2)}{2(0.03)^2} = -\frac{\ln(0.005)}{0.0018}.$$

Since $\ln(0.005) \approx -5.2983$,

$$n = \frac{5.2983}{0.0018} = 2943.5.$$

Hence,

$$n \geq 2944.$$

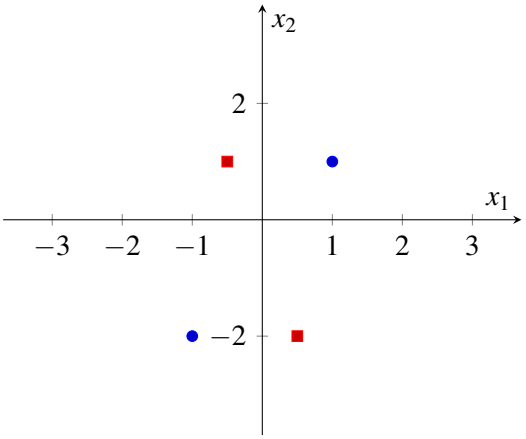
b) Consider the following data samples along with their ground truth values y. Answer the following questions:

$\mathbf{x_1}$	$\mathbf{x_2}$	$\mathbf{y}$
-0.5	1	-1
0.5	-2	-1
-1	-2	+1
1	1	+1

i) Plot the data on a Cartesian plane to visually demonstrate that the samples are non-linearly separable and cannot be classified with zero error using a linear classifier (e.g., a perceptron). **[5 marks]**

Solution:

By plotting the data sample points on Cartesian plane, we can easily observe that the two classes cannot be separated with a line.



ii) Apply the Pocket Algorithm using the **main Perceptron Learning Algorithm (PLA)** you learned in class for **5 weight updates** (not including the initialization).

At each update:

- Identify the **first misclassified sample** and use it to update the weights.

- After each update, **estimate the number of misclassified samples** on the entire dataset.

Finally, report the **pocket weight vector** $\mathbf{w}^{\text{poc}}$ and the corresponding **number of misclassified samples**. **[30 marks]**

Solution:

Include bias:

$$\mathbf{x}_1 = (1, -0.5, 1), \quad y_1 = -1,$$
$$\mathbf{x}_2 = (1, 0.5, -2), \quad y_2 = -1,$$
$$\mathbf{x}_3 = (1, -1, -2), \quad y_3 = +1,$$
$$\mathbf{x}_4 = (1, 1, 1), \quad y_4 = +1.$$

PLA update: $\mathbf{w} \leftarrow \mathbf{w} + y_i \mathbf{x}_i$. **Pocket:** keep the weight vector with the fewest training errors.

Initialization:

$$\mathbf{w}_{(1)} = (0,0,0), \qquad \text{pocket } \mathbf{w}^{\text{poc}} = \mathbf{w}_{(1)}.$$

Total errors with $\mathbf{w}_{(1)}$:

$$\text{sign}(\mathbf{w}_{(1)}^\top \mathbf{x}_i) = +1 \ \forall i \ \Rightarrow \ \text{errors on } i = 1,2 \Rightarrow E^{\text{poc}} = 2.$$

Update 1 (on $\mathbf{x}_1$):

$$\mathbf{w}_{(1)}^\top \mathbf{x}_1 = 0 \Rightarrow \hat{y}_1 = +1 \neq y_1 = -1 \text{ (misclassified)}$$

$$\mathbf{w}_{(2)} = \mathbf{w}_{(1)} + y_1 \mathbf{x}_1 = (0,0,0) + (-1)(1,-0.5,1) = (-1, 0.5, -1).$$

Errors with $\mathbf{w}_{(2)}$:

$$\hat{y}_1 = \text{sign}(-1 - 0.25 - 1) = -1 = y_1,$$
$$\hat{y}_2 = \text{sign}(-1 + 0.25 + 2) = +1 \neq y_2,$$
$$\hat{y}_3 = \text{sign}(-1 - 0.5 + 2) = +1 = y_3,$$
$$\hat{y}_4 = \text{sign}(-1 + 0.5 - 1) = -1 \neq y_4.$$

Thus 2 errors; not better than pocket ($E^{\text{poc}} = 2$).

Update 2 (on $\mathbf{x}_2$):

$$\mathbf{w}_{(2)}^\top \mathbf{x}_2 = -1 + 0.25 + 2 = 1.25 \Rightarrow \hat{y}_2 = +1 \neq y_2 = -1 \text{ (misclassified)}$$

$$\mathbf{w}_{(3)} = \mathbf{w}_{(2)} + y_2\mathbf{x}_2 = (-1, 0.5, -1) + (-1)(1, 0.5, -2) = (-2,\ 0,\ 1).$$

Errors with $\mathbf{w}_{(3)}$:

$$\begin{aligned}
\hat{y}_1 &= \text{sign}(-2+0+1) = -1 = y_1,\\
\hat{y}_2 &= \text{sign}(-2+0-2) = -1 = y_2,\\
\hat{y}_3 &= \text{sign}(-2+0-2) = -1 \neq y_3,\\
\hat{y}_4 &= \text{sign}(-2+0+1) = -1 \neq y_4.
\end{aligned}$$

Thus 2 errors; pocket unchanged.

Update 3 (on $\mathbf{x}_3$):

$$\mathbf{w}_{(3)}^\top \mathbf{x}_3 = -2+0-2 = -4 \Rightarrow \hat{y}_3 = -1 \neq y_3 = +1 \text{ (misclassified)}$$

$$\mathbf{w}_{(4)} = \mathbf{w}_{(3)} + y_3\mathbf{x}_3 = (-2, 0, 1) + (1, -1, -2) = (-1,\ -1,\ -1).$$

Errors with $\mathbf{w}_{(4)}$:

$$\begin{aligned}
\hat{y}_1 &= \text{sign}(-1+0.5-1) = -1 = y_1,\\
\hat{y}_2 &= \text{sign}(-1-0.5+2) = +1 \neq y_2,\\
\hat{y}_3 &= \text{sign}(-1+1+2) = +1 = y_3,\\
\hat{y}_4 &= \text{sign}(-1-1-1) = -1 \neq y_4.
\end{aligned}$$

Again 2 errors; pocket unchanged.

Update 4 (on $\mathbf{x}_2$):

$$\mathbf{w}_{(4)}^\top \mathbf{x}_2 = -1-0.5+2 = 0.5 \Rightarrow \hat{y}_2 = +1 \neq y_2 = -1 \text{ (misclassified)}$$

$$\mathbf{w}_{(5)} = \mathbf{w}_{(4)} + y_2\mathbf{x}_2 = (-1, -1, -1) + (-1)(1, 0.5, -2) = (-2,\ -1.5,\ 1).$$

Errors with $\mathbf{w}_{(5)}$:

$$\begin{aligned}
\hat{y}_1 &= \text{sign}(-2+0.75+1) = -1 = y_1,\\
\hat{y}_2 &= \text{sign}(-2-0.75-2) = -1 = y_2,\\
\hat{y}_3 &= \text{sign}(-2+1.5-2) = -1 \neq y_3,\\
\hat{y}_4 &= \text{sign}(-2-1.5+1) = -1 \neq y_4.
\end{aligned}$$

Still 2 errors; pocket unchanged.

Update 5 (on $\mathbf{x}_3$):

$$\mathbf{w}_{(5)}^\top \mathbf{x}_3 = -2+1.5-2 = -2.5 \Rightarrow \hat{y}_3 = -1 \neq y_3 = +1 \text{ (misclassified)}$$

$$\mathbf{w}_{(6)} = \mathbf{w}_{(5)} + y_3\mathbf{x}_3 = (-2,-1.5,1) + (1,-1,-2) = (-1,\,-2.5,\,-1).$$

Errors with $\mathbf{w}_{(6)}$:

$$\begin{aligned}
\hat{y}_1 &= \text{sign}(-1+1.25-1) = -1 = y_1,\\
\hat{y}_2 &= \text{sign}(-1-1.25+2) = -1 = y_2,\\
\hat{y}_3 &= \text{sign}(-1+2.5+2) = +1 = y_3,\\
\hat{y}_4 &= \text{sign}(-1-2.5-1) = -1 \neq y_4.
\end{aligned}$$

Now only 1 error; pocket improves.

Result after 5 updates:

$$\text{Current PLA weights } \mathbf{w}_{(6)} = (-1,\,-2.5,\,-1)$$

$$\text{Pocket weights } \mathbf{w}^{\text{poc}} = (-1,\,-2.5,\,-1), \quad \text{best error } E^{\text{poc}} = 1.$$

Hence, after 5 updates, the pocket algorithm achieves the best possible classification with only one misclassified sample.

iii) Apply the regression for classification approach by using closed form solution to find the weight vector. Based on the result, discuss whether this approach provides good initial weights for perceptron classifier.

[15 marks]

Solution:

Augment with bias and collect rows in

$$X = \begin{bmatrix} 1 & -0.5 & 1 \\ 1 & 0.5 & -2 \\ 1 & -1 & -2 \\ 1 & 1 & 1 \end{bmatrix}, \qquad y = \begin{bmatrix} -1 \\ -1 \\ +1 \\ +1 \end{bmatrix}.$$

Use the normal equations (closed form):

$$\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top y.$$

Compute $X^\top X$ and $X^\top y$.

$$X^\top X = \begin{bmatrix} 4 & 0 & -2 \\ 0 & 2.5 & 1.5 \\ -2 & 1.5 & 10 \end{bmatrix}, \qquad X^\top y = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Hence,

$$\hat{\mathbf{w}} = (X^\top X)^{-1} \cdot \mathbf{0} = \mathbf{0}\ .$$

The closed-form solution did not add much information and resulted in the same weight vector we use to initialize the PLA.

iv) Suggest a non-linear transformation $\Phi(\mathbf{x}) = (\mathbf{z}_1, \mathbf{z}_2)$ that will make the data linearly separable while maintaining the feature space dimension the same. Then, plot the transformed data on a Cartesian plane to visually demonstrate that the samples become linearly separable under the chosen transformation. **[20 marks]**

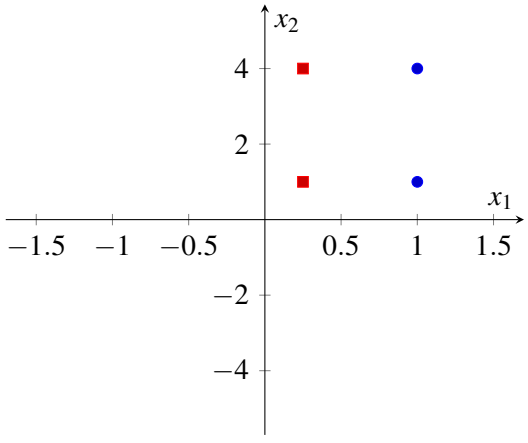
Solution: The following transformation will make the data linearly separable:

$$\Phi(\mathbf{x}) = (\mathbf{z}_1, \mathbf{z}_2) = (\mathbf{x}_1^2, \mathbf{x}_2^2)$$

The resulting dataset after the transformation is

$\mathbf{x_1}$	$\mathbf{x_2}$	$\mathbf{y}$
0.25	1	-1
0.25	4	-1
1	4	+1
1	1	+1

Plotting the data on Cartesian plane will result:



It shows clearly that the data became linealry seperable

2. Multilayer Perceptron, Neural Network, Regularization, Validation and VC Dimension

a) The data in the table below cannot be classified with zero error using 1-D perceptron

x	y
1	+1
2	−1
4	+1

Suppose that we will use instead a network with two hidden perceptrons in parallel, followed by an output perceptron. The hidden perceptrons implement thresholds at $x = 1.5$ and $x = 3$:

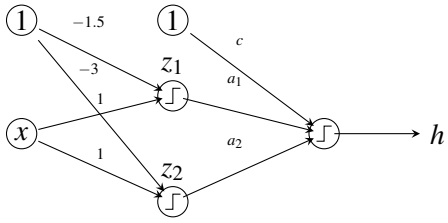
$$z_1 = \text{sign}(x - 1.5), \qquad z_2 = \text{sign}(x - 3).$$

The output perceptron combines them as

$$h = \text{sign}(a_1 z_1 + a_2 z_2 + c).$$

i) Draw a schematic diagram of the two-layer perceptron described in the question. Label all nodes and connections clearly. **[7 marks]**

Solution:



ii) For each data point $x = 1, 2, 4$, compute the hidden activations (z_1, z_2).

[8 marks]

Solution: *Hidden activations.*

x	$z_1 = \text{sign}(x - 1.5)$	$z_2 = \text{sign}(x - 3)$
1	−1	−1
2	+1	−1
4	+1	+1

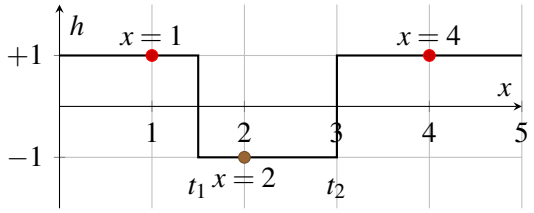
iii) Find values of a_1, a_2, c such that the network outputs the correct labels. Then plot h for $x \in [0,5]$. **[15 marks]**

Solution: *Output weights.* One valid choice is

$$a_1 = -1, \qquad a_2 = 2, \qquad c = 2.$$

Check:

$$x = 1: \ \text{sign}(-1(-1) + 2(-1) + 2) = \text{sign}(1 - 2 + 2) = +1,$$
$$x = 2: \ \text{sign}(-1(1) + 2(-1) + 2) = \text{sign}(-1 - 2 + 2) = -1,$$
$$x = 4: \ \text{sign}(-1(1) + 2(1) + 2) = \text{sign}(-1 + 2 + 2) = +1.$$



iv) In the $(z_1, z_2)-$plane plot the points representing the hidden-layer activations for the three input samples. Using your chosen weights, find the decision boundary equation and sketch the decision boundary of the output perceptron, indicating which side corresponds to each class.

[12 marks]

Solution:

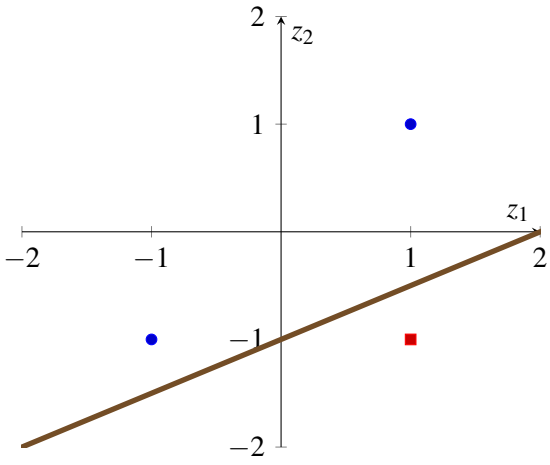
Based on the chosen weights, the decision boundary equation will become as follows:

$$-\mathbf{z}_1 + 2\mathbf{z}_2 + 2 = 0$$

Solving for $\mathbf{z}_2$:

$$\mathbf{z}_2 = \frac{1}{2}\mathbf{z}_1 - 1$$

Plotting:



v) Conclude that with this two-layer network, three points on the real line can be shattered. What does this imply about the VC dimension of this architecture compared to that of a single 1D perceptron? **[8 marks]**

Solution: The three inputs map to distinct hidden-space points $(-1,-1),(1,-1),(1,1)$, which are non-collinear. A single linear output unit can realize any labeling on three non-collinear points; hence this two-layer network can shatter three points on $\mathbb{R}$ (so $d_{VC} \geq 3$), strictly larger than that of a single 1D perceptron ($d_{VC} = 2$).

b) We are interested in a text classification task. Specifically, given a sentence represented [1] by a feature vector $x \in \mathbb{R}^d$, its corresponding label y is $+1$ if the sentence is grammatically correct, and -1 otherwise. A colleague of yours told you about this very recent paper[2] to appear in *NeurIPS 2025*, where a kernel based framework was proposed that leverages a frozen pretrained large language models to generate support vectors and corresponding labels for any given classification task.

i) You used the proposed framework in the paper for the given task and were able to generate n support vectors and corresponding labels $(\hat{x}_1,\hat{y}_1),\ldots,(\hat{x}_n,\hat{y}_n),\in \mathbb{R}^d \times \{-1,+1\}$ for your task. However, the method only outputs noisy version of the true support vectors and corresponding labels $(x_1,y_1),\ldots,(x_n,y_n),\in \mathbb{R}^d \times \{-1,+1\}$. More pre-

[1] For simplicity, we assume that sentences have been embedded in $\mathbb{R}^d$ through some feature transformation

[2] **[Paper]** S. Ohsawa, Support Vector Generation: Kernelizing Zero-Shot Classifiers from Pre-Trained Language Models, NeurIPS 2025

cisely, for all $i \in [n]$,

$$\hat{x}_i \sim \mathcal{N}(x_t, \sigma^2 I_d) \quad \text{and} \quad \hat{y}_i = \begin{cases} y_t & \text{with probability} \quad 1 - p(\hat{x}_i) \\ -y_t & \text{with probability} \quad p(\hat{x}_i) \end{cases}$$

where $\sigma > 0$ and $p(x_i) \in (0,1)\backslash\{1/2\}$. Recall that x_i is a support vector with label y_i if and only if $y_i = w_\star^\top x_t + b_\star$. Show that

$$\hat{y}_i = \begin{bmatrix} w_\star \\ b_\star \end{bmatrix}^\top z_i + \varepsilon_i \tag{2.1}$$

where z_i can only be expressed in terms of $\hat{x}_i$ and $p(\hat{x}_i)$, and ε_i can only be expressed in terms of $y_i, \hat{y}_i, x_i, \hat{x}_i, p(\hat{x}_i), w_\star$ and satisfies $\mathbb{E}[\varepsilon_i|\hat{x}_i] = 0$.

[15 marks]

Hint: Verify that $\mathbb{E}[\hat{y}_i|\hat{x}_i] = \alpha y_i$, where α only depends on $p(\hat{x}_i)$.

Solution: Note that $\mathbb{E}[\hat{y}_i|\hat{x}_i] = (1 - 2p_i(x_i))y_i$. We have

$$\begin{aligned}
\hat{y}_i &= (1-2p(\hat{x}_i))y_i = \hat{y}_i - (1-2p(\hat{x}_i))y_i \\
&= (1-2p(\hat{x}_i))w_\star^\top x_t + (1-2p(\hat{x}_i))b_\star + \hat{y}_i - (1-2p(\hat{x}_i))y_i \\
&= (1-2p(\hat{x}_i))w_\star^\top \hat{x}_t + (1-2p(\hat{x}_i))b_\star + \hat{y}_i - (1-2p(\hat{x}_i))y_i + (1-2p(\hat{x}_i))w_\star^\top (x_t - \hat{x}_t)
\end{aligned}$$

We see that z_i and ε_i can be defined as

$$\begin{aligned}
z_i &= \begin{bmatrix} (1-2p(\hat{x}_i))\hat{x}_i \\ (1-2p(\hat{x}_i)) \end{bmatrix} \\
\varepsilon_i &= \hat{y}_i - (1-2p(\hat{x}_i))y_i + (1-2p(\hat{x}_i))w_\star^\top (x_t - \hat{x}_t).
\end{aligned}$$

Notice that

$$\mathbb{E}[\varepsilon_i|\hat{x}_i] = \mathbb{E}[\hat{y}_i - (1-2p(\hat{x}_i))y_i|\hat{x}_i] + (1-2p(\hat{x}_i))w_\star^\top \mathbb{E}[(x_t - \hat{x}_t)|\hat{x}_i] = 0$$

ii) Assume knowledge of $p(\hat{x}_i)$ for all i. Using $(z_1, \hat{y}_1), \ldots, (z_n, \hat{y}_n)$ formulate the least squares problem. Consider the following matrix notation

$$Z^\top = \begin{bmatrix} z_1 & \ldots & z_n \end{bmatrix} \in \mathbb{R}^{d+1 \times n} \quad \text{and} \quad \hat{y} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_n \end{bmatrix} \in \mathbb{R}^n,$$

and write down the unconstrained solution the least squares problem. Denote such solution by $(\hat{w}, \hat{b})$, and assuming that $(x_1, y_1), \ldots, (x_n, y_n)$

are independent verify that it is an unbiased solution of $w_\star, b_\star$. **[15 marks]**

Solution: The formulation of the least squares problem is as follows:

$$\min_{w,b} \sum_{i=1}^{n} \left| y_i - \begin{bmatrix} w \\ b \end{bmatrix}^\top z_i \right|^2 .$$

The solution to the unconstrained Least squares problem has a closed form solution:

$$\begin{bmatrix} \hat{w} \\ \hat{b} \end{bmatrix} = (Z^\top Z)^{-1} Z^\top \hat{y}$$

Note that

$$\hat{y} = Z \begin{bmatrix} w_\star \\ b_\star \end{bmatrix} + E, \qquad \text{with} \qquad E = \begin{bmatrix} \varepsilon_1 \\ \vdots, \\ \varepsilon_n, \end{bmatrix}$$

Thus, we have

$$\begin{aligned} \mathbb{E}\left[\begin{bmatrix} \hat{w} \\ \hat{b} \end{bmatrix} | \hat{x}_1, \ldots, \hat{x}_n \right] &= \mathbb{E}\left[(Z^\top Z)^{-1} Z^\top \hat{y} | \hat{x}_1, \ldots, \hat{x}_n \right] \\ &= \mathbb{E}\left[(Z^\top Z)^{-1} Z^\top \left(Z \begin{bmatrix} w_\star \\ b_\star \end{bmatrix} + E \right) | \hat{x}_1, \ldots, \hat{x}_n \right] \\ &= \begin{bmatrix} w_\star \\ b_\star \end{bmatrix} + (Z^\top Z)^{-1} Z^\top \mathbb{E}[E | \hat{x}_1, \ldots, \hat{x}_n] \\ &= \begin{bmatrix} w_\star \\ b_\star \end{bmatrix} \end{aligned}$$

iii) Your colleague believes that the $w_\star$ should be sparse. Suggest an approach to incorporate this belief into your minimization problem. Write the corresponding minimization problem with a regularization parameter λ. Your colleague believes that $\lambda \in \{1, 10, 100\}$, discuss what should be the best practice to find the best regularization parameter. **[12 marks]**

Solution: Use ℓ_1 penalty and write down the minimization problem as

$$\min_{w,b} \sum_{i=1}^{n} \left| y_i - \begin{bmatrix} w \\ b \end{bmatrix}^\top z_i \right|^2 + \lambda \|w\|_1 .$$

To select λ, we may use cross validation. For example, we split our training data into two sets, on the first we solve our minimization problem and on the second we evaluate the solution. We do this for every $\lambda \in \{1, 10, 100\}$.

iv) In the two previous questions, you have been using a regression formulation to solve a classification task. Is this a good idea in general? Is there a better idea? **[8 marks]**

Solution: This can indeed provide a good initialization which can reduce the number of iterations for learning a good classifier but in general the squared loss can be sensitive to outliers and thus may not be always a good idea. A better idea would be to use logistic regression which known to be more robust to outliers.

3. Support vector machines, kernel methods

a) You are given a dataset containing n input-label pairs, $(x_1, y_1), \ldots, (x_n, y_n) \in \mathscr{X} \times \{-1, 1\}$ where $\mathscr{X} \subseteq \mathbb{R}^d$. You goal is to learn a hypothesis $h : \mathscr{X} \mapsto \{-1, 1\}$ that fits the data from a class of hypothesis $\mathscr{H}$.

i) A colleague of yours suggests using an SVM to solve the problem. Define the hypothesis class $\mathscr{H}$ you use, and describe under what condition using SVMs is a good idea. **[10 marks]**

Solution: The hypothesis class we use in SVM is that of hyperplanes $\mathscr{H} = \{h : x \mapsto w^\top x + b; w \in \mathbb{R}^d, b \in \mathbb{R}\}$. It is a good idea to use SVMs if the data is separable.

ii) For a given hypothesis in $h \in \mathscr{H}$, an input-label pair (x, y), the hinge loss between $h(x)$ and y is defined as $\ell_{\text{hinge}}(h(x), y) = \max(1 - yh(x), 0)$. Your colleague suggests to formulate the binary classification problem as follows:

$$\min_{h \in \mathscr{H}} \sum_{i=1}^{n} \ell_{\text{hinge}}(h(x_i), y_i) + R(h) \tag{3.1}$$

where $R(\cdot)$ is some regularization function. Show that with a proper choice of $R(\cdot)$, the resulting problem is equivalent to a soft-margin SVM problem formulation with parameter C. **[20 marks]**

Solution: Let us choose $R(h) = \frac{1}{C}\|w\|_2^2$

$$\min_{w,b} \frac{1}{2C}\|w\|_2^2 + \sum_{i=1}^{n} \max(1 - y_i(x_i^\top w + b), 0)$$

Introduce the slack variables $\xi_i \in \mathbb{R}$, and notice that

$$\xi_i \geq \max(1 - y_i(x_i^\top w + b), 0) \iff 1 - y_i(x_i^\top w + b) \leq \xi_i \quad \text{and} \quad \xi_i \geq 0$$

Therefore, we have for all w, b,

$$\begin{aligned} \frac{1}{2C}\|w\|_2^2 + \sum_{i=1}^{n} \max(1 - y_i(x_i^\top w + b), 0) = \min_{\xi} \frac{1}{2C}\|w\|_2^2 + \sum_{i=1}^{n} \xi_i \\ s.t. \quad 1 - y_i(x_i^\top w + b) \leq \xi_i \\ \text{and} \quad \xi_i \geq 0. \end{aligned}$$

Thus,

$$\min_{w,b} \frac{1}{2C}\|w\|_2^2 + \sum_{i=1}^{n} \max(1 - y_i(x_i^\top w + b), 0) = \min_{w,b,\xi} \frac{1}{2C}\|w\|_2^2 + \sum_{i=1}^{n} \xi_i$$

$$s.t. \quad 1 - y_i(x_i^\top w + b) \leq \xi_i$$
$$\text{and} \quad \xi_i \geq 0$$

iii) Regarding the hinge loss formulation:

A. Let $s = -yh(x)$. Plot the hinge loss as a function of s (i.e. plot the function $s \mapsto \ell_{\text{hinge}}(h(x), y)$), when s varies in $(-4, 4)$.

B. For a given (x, y), what is the gradient of $\ell_{\text{hinge}}(h(x), y)$ with respect to the parameters defining your hypothesis h.

C. What benefits do you see in using the formulation (3.1)?

[15 marks]

Solution: A. The plot is exactly as the one in the slide about the shifted hinge loss from lecture 2 except with the shift.

B. The gradients (or more precisely the sub-gradients) are given as

$$\nabla_w \ell_{\text{hinge}}(h(x), y) = \begin{cases} -yx & \text{if } 1 - y(w^\top x + b) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\nabla_b \ell_{\text{hinge}}(h(x), y) = \begin{cases} -y & \text{if } 1 - y(w^\top x + b) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

C. The problem formulations (3.1) has the advantage of being an unconstrained and convex problem. Thus we may use directly gradient descent to solve the problem. This provides an alternative way to solve the soft-margin SVM problem.

iv) Recall the soft-margin SVM formulation with parameter C:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n} \xi_i$$
$$s.t. \quad y_i(x_i^\top w + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0.$$

Write down the Lagrangian of this problem and corresponding equations from the KKT condition and derive the corresponding Lagrange dual problem. **[15 marks]**

Solution: The Lagrangian corresponding to your problem is given by

$$\mathscr{L}(w,b,\xi,\alpha,\beta) = \frac{\|w\|^2}{2} + C\sum_{i=1}^{n}\xi_i + \sum_{i=1}^{n}\alpha_i(1-\xi_i - y_i(w^\top x_i + b) - \sum_{i=1}^{n}\beta_i\xi_i, \tag{3.2}$$

where α_i (resp. β_i) correspond to the Lagrange multiplier of the inequality $1 - y_i(w^\top x_i + b) - \xi \leq 0$ (resp. $-\xi_i \leq 0$) and must verify $\alpha_i, \beta_i \geq 0$. The KKT conditions yield

$$\nabla\mathscr{L}_w = w - \sum_{i=1}^{n}\alpha_i y_i x_i = 0 \implies w = \sum_{i=1}^{n}\alpha_i y_i x_i \tag{3.3}$$

$$\nabla\mathscr{L}_b = -\sum_{i=1}^{n}\alpha_i y_i = 0 \implies \sum_{i=1}^{n}\alpha_i y_i = 0 \tag{3.4}$$

$$\nabla_{\xi_i}\mathscr{L} = C - \alpha_i - \beta_i = 0 \implies \alpha_i + \beta_i = C \tag{3.5}$$

$$\alpha_i(1-\xi_i - y_i(w^\top x_i + b)) = 0 \implies (1-\xi_i - y_i(w^\top x_i + b)) = 0 \quad \text{or} \quad \alpha_i = 0 \tag{3.6}$$

$$\beta_i\xi_i = 0 \implies \beta_i = 0 \quad \text{or} \quad \xi_i = 0 \tag{3.7}$$

Note that using the above equations yields the finial maximization problem over α

$$\max_{\alpha}\mathscr{L}(\alpha) = \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}y_i y_j \alpha_i \alpha_i^\top x_j$$

subject to $0 \leq \alpha_i \leq C, \sum_{i=1}^{n}\alpha_i y_i = 0$. To see that note that plugging in equation (3.5) in the Lagrangian (3.2), we obtain

$$\mathscr{L}(w,b,\xi,\alpha,\beta) = \frac{\|w\|^2}{2} + \sum_{i=1}^{n}\alpha_i(1 - y_i(w^\top x_i + b),$$

Next using (3.4) in the above expression, we obtain

$$\mathscr{L}(w,b,\xi,\alpha,\beta) = \frac{\|w\|^2}{2} + \sum_{i=1}^{n}\alpha_i(1 - y_i w^\top x_i,$$

Finally, using the expression of w given in (3.3), we obtain

$$\mathscr{L}(w,b,\xi,\alpha,\beta) = \mathscr{L}(\alpha) = \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}y_i y_j \alpha_i \alpha_i^\top x_j$$

Now notice that (3.5) with the fact that $\alpha_i, \beta_i \geq 1$ imply that $1 \leq \alpha_i \leq C$. Recalling the condition (3.4), we recognize the presented dual formulation.

v) An oracle informed you that $x_{i_1}, \ldots, x_{i_{d+1}}$ are support vectors and that the matrix

$$\begin{bmatrix} x_{i_1} & \ldots & x_{i_{d+1}} \\ 1 & \ldots & 1 \end{bmatrix} \in \mathbb{R}^{d+1 \times d+1}$$

is full rank. Propose a solution to the problem that leverages this additional information, by explicitly expressing of the solution w and b. Give one benefit and one shortcoming to using this solution. **[24 marks]**

Solution: Since $x_{i_1}, \ldots, x_{i_{d+1}}$ are support vectors, they must satisfy

$$y_{i_k} = w_\star^\top x_{i_k} + b_\star = \begin{bmatrix} x_{i_k} \\ 1 \end{bmatrix}^\top \begin{bmatrix} w_\star \\ b_\star \end{bmatrix}$$

which gives the matrix linear equation with unknown $w_\star, b_\star$.

$$\begin{bmatrix} y_{i_1} \\ \vdots \\ y_{i_{d+1}} \end{bmatrix} = \begin{bmatrix} x_{i_1} & \ldots & x_{i_{d+1}} \\ 1 & \ldots & 1 \end{bmatrix}^\top \begin{bmatrix} w_\star \\ b_\star \end{bmatrix}.$$

This implies

$$\begin{bmatrix} w_\star \\ b_\star \end{bmatrix} = \left(\begin{bmatrix} x_{i_1} & \ldots & x_{i_{d+1}} \\ 1 & \ldots & 1 \end{bmatrix}^\top \right)^{-1} \begin{bmatrix} y_{i_1} \\ \vdots \\ y_{i_{d+1}} \end{bmatrix}$$

One possible benefit to having knowledge of the support vectors is that the above solution can be computationally more efficient. Note that the proposed solution only requires inverting a d^2 matrix which typically requires $O(d^3)$ operation while using a solver for the soft-margin may require at least of order $O(n)$. So when $n \gg d^3$, the proposed method is better.

One shortcoming to the above solution is that if my data is not linearly separable, then knowledge of these support vectors does not provide any advantage once we use a kernel method for the classification task. To see that, notice that if $x_{i_1}, \ldots, x_{i_d+1}$ are support vectors in the original problem, after using a nonlinear transformation ϕ, the new feature vectors $\phi(x_{i_1}), \ldots, \phi(x_{i_{d+1}})$ are not necessarily feature vectors in the new formulation. The kernel trick will not work.

vi) You run the obtained solution on some test data, but you noticed that the test error is significantly larger than the training error. List two

possible issues that could explain this poor performance, and propose a solution for each one of them. **[16 marks]**

Solution: One possible issue could be that the choice of C is not good. One solution to this issue would be to use cross-validation and to select the best C among a set of possible values of C. A second possible issue, could be that your data is not linearly separable in the considered features. A solution to this issue would be to use a kernel method. One typical choice of kernels to use is the radial basis kernel.

4. Principal components analysis, Clustering, GMMs

a) You are given three 2D points:

$$x_1 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \quad x_3 = \begin{bmatrix} -2 \\ 0 \end{bmatrix}.$$

i) Compute the sample mean μ and the mean-centred dataset $\tilde{x}_i = x_i - \mu$.

[3 marks]

Solution:

$$\mu = \tfrac{1}{3}(x_1 + x_2 + x_3) = \begin{bmatrix} 0 \\ \frac{2}{3} \end{bmatrix}, \quad \tilde{x}_1 = \begin{bmatrix} 2 \\ -\frac{2}{3} \end{bmatrix}, \tilde{x}_2 = \begin{bmatrix} 0 \\ \frac{4}{3} \end{bmatrix}, \tilde{x}_3 = \begin{bmatrix} -2 \\ -\frac{2}{3} \end{bmatrix}.$$

ii) Compute the empirical covariance matrix $S = \frac{1}{3}\sum_{i=1}^{3} \tilde{x}_i \tilde{x}_i^\top$.

[6 marks]

Solution:

$$S = \begin{bmatrix} \frac{8}{3} & 0 \\ 0 & \frac{8}{9} \end{bmatrix}.$$

iii) Find eigenvalues/eigenvectors of S. Identify the first principal component u_1 (unit vector). **[5 marks]**

Solution:

$$\lambda_1 = \tfrac{8}{3}, \quad \lambda_2 = \tfrac{8}{9}, \quad u_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, u_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The first principal component is u_1

iv) Compute 1-D PCA coordinates $z_i = u_1^\top \tilde{x}_i$ and rank-1 reconstructions $\hat{x}_i = \mu + z_i u_1$. Briefly interpret $z_i u_1$. **[6 marks]**

Solution:

$$z_1 = 2, \quad z_2 = 0, \quad z_3 = -2.$$

$$\hat{x}_1 = \begin{bmatrix} 2 \\ \frac{2}{3} \end{bmatrix}, \hat{x}_2 = \begin{bmatrix} 0 \\ \frac{2}{3} \end{bmatrix}, \hat{x}_3 = \begin{bmatrix} -2 \\ \frac{2}{3} \end{bmatrix}.$$

Each $z_i u_1$ is the projection of $\tilde{x}_i$ onto the first PC axis.

v) If you kept only u_2, what fraction of total variance is captured? Comment. **[4 marks]**

Solution:

Total variance $= \lambda_1 + \lambda_2 = \frac{32}{9}$, Captured by $u_2 = \frac{8}{9}$, Fraction $= \frac{1}{4} = 25\%$.

Keeping u_2 alone loses 75% of the variance

vi) Why would we want to reduce the dimensionality of our data? Can you think of examples where this can be useful? **[4 marks]**

Solution: There can be several reasons: i) Fewer features are easier to interpret: we might want to know why our model outputs a certain diagnosis, and only some of the patient record details will be relevant. ii) Models with fewer features are easier to handle computationally. iii) Our data might be arbitrarily high-dimensional because of noise, so we would like to access the lower-dimensional information-carrying signal from the data.

vii) Outline the PCA workflow for a K-dimensional representation and how to transform/reconstruct points. **[12 marks]**

Solution:

- Center data: $\mu = \frac{1}{n}\sum_i x_i$, $\tilde{x}_i = x_i - \mu$.
- Compute covariance $S = \frac{1}{n}\sum_i \tilde{x}_i \tilde{x}_i^\top$ (or use SVD).
- Choose target K using scree plot or variance threshold.
- Compute top-K eigenpairs (λ_j, u_j).
- Form $U_K = [u_1, \ldots, u_K]$.
- Low-dimensional coordinates: $z_i = U_K^\top \tilde{x}_i$.
- Reconstruction: $\hat{x}_i = \mu + U_K z_i$.
- Explained variance ratio: $\frac{\sum_{j=1}^K \lambda_j}{\sum_{j=1}^D \lambda_j}$.

b) Consider nine 1D points: $\{0, 1, 2, 6, 7, 8, 20, 21, 22\}$. Use Euclidean distance $d(x, x') = |x - x'|$ and merge one pair per step, breaking ties by lower x first. Compute the clusters, using single-linkage first, followed by max-linkage distance. **[20 marks]**

Solution:

Single linkage:

i) $\{0\}$–$\{1\}$ @1 $\rightarrow \{0,1\}$

ii) $\{0,1\}$–$\{2\}$ @1 $\rightarrow \{0,1,2\}$

iii) $\{6\}$–$\{7\}$ @1 $\rightarrow \{6,7\}$

iv) $\{6,7\}$–$\{8\}$ @1 $\rightarrow \{6,7,8\}$

v) $\{20\}$–$\{21\}$ @1 $\rightarrow \{20,21\}$

vi) $\{20,21\}$–$\{22\}$ @1 $\rightarrow \{20,21,22\}$

vii) $\{0,1,2\}$–$\{6,7,8\}$ @4 $\rightarrow \{0,1,2,6,7,8\}$

viii) previous – $\{20,21,22\}$ @12 $\rightarrow$ all points.

Complete linkage:

i) $\{0\}$–$\{1\}$ @1 $\rightarrow \{0,1\}$

ii) $\{6\}$–$\{7\}$ @1 $\rightarrow \{6,7\}$

iii) $\{20\}$–$\{21\}$ @1 $\rightarrow \{20,21\}$

iv) $\{0,1\}$–$\{2\}$ @2 $\rightarrow \{0,1,2\}$

v) $\{6,7\}$–$\{8\}$ @2 $\rightarrow \{6,7,8\}$

vi) $\{20,21\}$–$\{22\}$ @2 $\rightarrow \{20,21,22\}$

vii) $\{0,1,2\}$–$\{6,7,8\}$ @8 $\rightarrow \{0,1,2,6,7,8\}$

viii) previous – $\{20,21,22\}$ @22 $\rightarrow$ all points.

(Students should draw dendrograms with these merge heights.)

c) Consider k-means clustering on a set of datapoints.

i) Write the k-means objective with Euclidean distance. **[5 marks]**

Solution: With $r_{nj} \in \{0,1\}$ and $\sum_j r_{nj} = 1$:

$$\min_{\{\mu_j\},\{r_{nj}\}} \sum_{n=1}^{N} \sum_{j=1}^{k} r_{nj} \|x_n - \mu_j\|^2.$$

ii) State the two Lloyd steps that monotonically decrease the objective.

[5 marks]

Solution:

- **Assignment:** assign each x_n to its nearest centroid.

- **Update:** $\mu_j = \dfrac{\sum_n r_{nj} x_n}{\sum_n r_{nj}}$ for non-empty clusters.

iii) Give the per-iteration time complexity in terms of numer of samples N,clusters k and number of features$D$. **[5 marks]**

Solution: Assignment: $O(NkD)$; update: $O(ND)$ (plus $O(kD)$). Overall per iteration: $O(NkD)$.

iv) Explain why k-means fails on two concentric rings and give a remedy. **[5 marks]**

Solution: k-means prefers convex, spherical clusters (Voronoi partitions). Two rings are non-convex, so clusters slice radially and mix rings. Remedies: spectral clustering, kernel k-means, or DBSCAN.

d) Consider a K-component GMM in $\mathbb{R}^D$:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \quad \pi_k \geq 0, \quad \sum_k \pi_k = 1.$$

i) Write the complete-data log-likelihood using latent one-hot variables $z_{nk} \in \{0,1\}$ with $\sum_k z_{nk} = 1$. **[4 marks]**

Solution:

$$\log p(\{x_n, z_n\}_{n=1}^N) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \left(\log \pi_k + \log \mathcal{N}(x_n \mid \mu_k, \Sigma_k)\right).$$

ii) Derive the E-step updates $\gamma_{nk} = \mathbb{E}[z_{nk}]$ **[6 marks]**

Solution:

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n \mid \mu_j, \Sigma_j)}.$$

iii) Derive the M-step updates for π_k, μ_k, Σ_k using $N_k = \sum_n \gamma_{nk}$. **[6 marks]**

Solution:

$$\pi_k^{\text{new}} = \frac{N_k}{N}, \qquad \mu_k^{\text{new}} = \frac{1}{N_k} \sum_n \gamma_{nk} x_n, \qquad \Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_n \gamma_{nk} (x_n - \mu_k^{\text{new}})(x_n - \mu_k^{\text{new}})^\top.$$

iv) State when GMMs reduce to k-means and give one advantage of GMMs. **[4 marks]**

Solution: If all $\Sigma_k = \sigma^2 I$ and $\sigma^2 \to 0$, MAP assignments reduce to nearest-centroid k-means. GMMs handle elliptical clusters and yield soft (probabilistic) assignments, enabling likelihood-based model selection (e.g., BIC).